COMP4601 Final Report

Shuruthy Dhushiyandan, Kaira Dumasia, Gilbert Tong
z5479689, z5441876, z5421556

1 Introduction

FPGAs present a compelling platform for embedded neural network inference, offering configurable spatial parallelism that general-purpose processors cannot match. Unlike CPUs, which execute instructions sequentially on fixed hardware, FPGAs allow compute to be structured around the specific data flow of an algorithm, enabling many operations to occur simultaneously within tight power and area budgets.

This project implements and accelerates a LeNet-inspired Convolutional Neural Network for MNIST handwritten digit classification on the AMD Kria KV260 FPGA using Xilinx Vitis High-Level Synthesis. The network is developed from an unoptimised sequential baseline through three successive hardware variants, with each optimisation guided by per-layer profiling. The central argument is that effective hardware acceleration requires identifying where latency actually resides before committing resources, as spending those resources on the wrong layer can cost more than it gains.

The final design, measured on-board against an ARM Cortex-A53 software implementation of the same network, achieves a 114.87× reduction in inference latency, from 19,392.64 μs to 168.82 μs per image, while maintaining 100% classification accuracy across both floating-point and fixed-point datapaths and fitting within 77% of the KV260's available LUTs.

2 Background and Context

The network used in this project is a reduced LeNet-5, originally developed by LeCun et al. for handwritten digit recognition. It processes 28×28 greyscale images through two convolutional layers with ReLU activation and max-pooling, followed by two fully-connected layers that produce 10 class scores. With approximately 45,000 parameters, all weights fit in on-chip BRAM, meaning performance is limited by compute rather than memory access.

Vitis HLS compiles C++ into hardware for Xilinx FPGAs, allowing the designer to express parallelism through pragmas. The three pragmas central to this project are `PIPELINE`, which overlaps loop iterations in time; `UNROLL`, which runs multiple iterations simultaneously by duplicating hardware; and `ARRAY_PARTITION`, which splits arrays across separate memory banks to allow parallel reads and writes.

A key synthesis metric is the Initiation Interval (II), which defines how many clock cycles must pass before the next loop iteration can begin, where II = 1 is optimal. At baseline, Conv2 and FC1 stall at II = 9 because floating-point addition has a multi-cycle latency, creating a read-after-write dependency on the accumulator that prevents back-to-back execution. Switching to `ap_fixed<20,8>` fixed-point arithmetic resolves this, as fixed-point addition completes in a single cycle and enables II = 1. Fixed-point multipliers also consume far less FPGA fabric, allowing higher unroll factors within the device resource budget. A wider `ap_fixed<40,16>` accumulator is used to prevent overflow during accumulation, while C-simulation retains floating-point arithmetic for correctness verification.

The target platform is the Kria KV260, which combines an ARM Cortex-A53 processor with Xilinx UltraScale+ programmable logic containing 117,120 LUTs, 1,248 DSP slices, and 144 BRAMs, clocked at 200 MHz.

3 System Architecture

The system is divided across two components of the Kria KV260. The Programmable Logic (PL) runs the CNN accelerator kernel synthesised by Vitis HLS at 200 MHz, and the Processing System (PS) runs an ARM Cortex-A53 host application that manages inference. The two components communicate over AXI4, with an `m_axi` master port transferring the input image and output logits to and from DDR memory, and an `s_axilite` slave port handling kernel control signals. Figure 1 illustrates this interface. The top-level HLS function `cnn()` accepts a `float[1][28][28]` input and writes a `float[10]` logit array, with all nine computational stages executing sequentially within this single kernel.

Layer Pipeline

Table 1 summarises the layer pipeline, output dimensions, and acceleration status for each stage.

Figure 2 shows the same pipeline as a dataflow diagram, with the three accelerated stages highlighted to make clear how little of the total pipeline required custom HLS treatment.

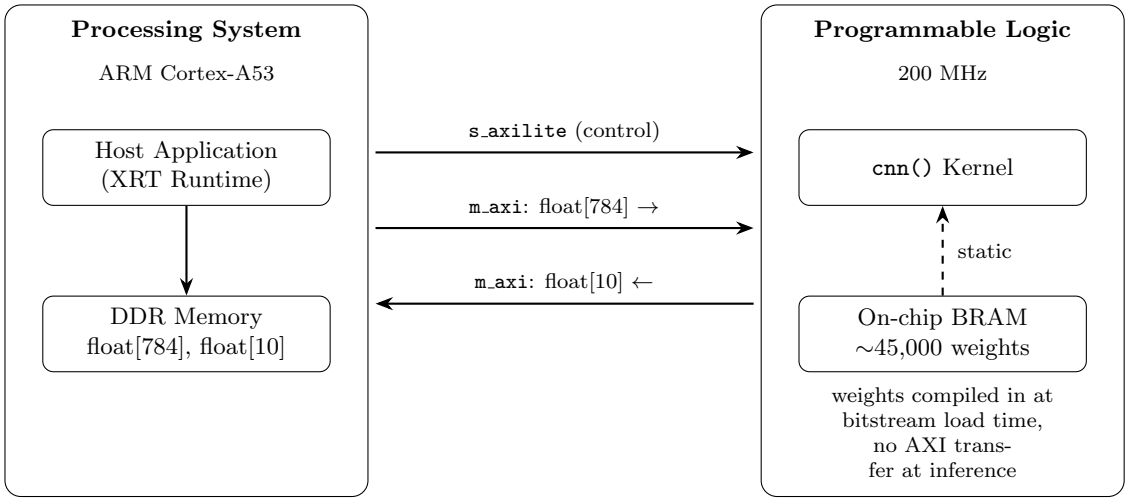

Figure 1: AXI interface between the Processing System and Programmable Logic.

Layer	Output Shape	Accelerated
Padding 1	$1 \times 32 \times 32$	No
Conv1 + ReLU	$3 \times 28 \times 28$	Yes
MaxPool 1	$3 \times 14 \times 14$	No
Padding 2	$3 \times 18 \times 18$	No
Conv2 + ReLU	$6 \times 14 \times 14$	Yes
MaxPool 2	$6 \times 7 \times 7$	No
Flatten	294	No
FC1 + ReLU	147	Yes
FC2	10	No

Table 1: Layer pipeline with output shapes and acceleration status.

Convolutional Layers

Both convolutional layers apply the same acceleration technique. Weights and inputs are copied into on-chip BRAM at the start of each call, replacing variable-latency DDR reads with single-cycle accesses. All multiplications per output pixel are computed simultaneously, with 25 taps for Conv1 and 75 for Conv2, and the resulting products are reduced through a balanced binary adder tree in $\lceil \log_2 N \rceil$ stages. This structure allows the output pixel loop to sustain $\mathrm{II} = 1$.

Fully-Connected Layers

FC1 computes a 294×147 matrix-vector product and was the dominant bottleneck at baseline, contributing approximately 389,000 cycles at $\mathrm{II} = 9$. In the accelerated design, `ARRAY_PARTITION` with cyclic factor 16 splits the input and weight arrays into 16 interleaved memory banks, enabling 16 values to be read simultaneously each cycle. The inner MAC loop is unrolled by the same factor, running 16 multiply-accumulate operations in parallel and reducing the inner loop to $\lceil 294/16 \rceil = 19$ pipelined iterations per neuron. FC1 consequently drops from approximately 389,000 cycles to approximately 4,000 cycles (per `csynth.rpt`), the remainder above the 19×147 iteration count coming from pipeline fill/drain and loop overhead.

Memory and Host Application

All approximately 45,000 trained weights are compiled as static constant arrays into the HLS kernel and stored in on-chip BRAM at bitstream load time. No weight data crosses the AXI bus during inference; the only bus traffic per inference is the 784-float input image and the 10-float logit array.

The host application uses the XRT runtime to manage the FPGA device and kernel. In accuracy mode, it classifies 100 bundled MNIST test images and reports accuracy and average inference time alongside a software reference run on the ARM core. In custom image mode, it reads a user-provided 28×28 image, applies MNIST normalisation, and returns per-class softmax probabilities alongside the predicted digit. This was the mode used during the live demonstration.

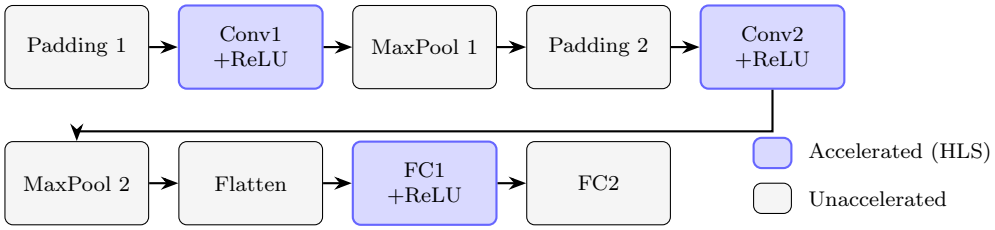

Figure 2: Nine-stage inference pipeline within the `cnn()` kernel. The three accelerated stages (Conv1, Conv2, FC1) account for the overwhelming majority of baseline latency; the six unaccelerated stages are cheap enough at these tensor sizes to leave as simple sequential C++.

4 Methodology

The acceleration effort followed a profiling driven loop rather than an up-front commitment of resources. What we did was profile the current design, identify the single largest contributor to latency, apply the minimal HLS transformation that removes it and then re-verify correctness and re-measure. This particular methodology was deliberate as we were able to minimise making the most expensive mistake, which is spending fabric on a stage that is not on the critical path, and the only defence against it is measurement.

Design flow

Each iteration passed through three evaluation stages of increasing cost:

1. **C-simulation** compiled the kernel with a host `g++` toolchain, where HLS pragmas are ignored and the code runs as sequential C++. This validated functional correctness against the 100-image MNIST test set in seconds, catching logic errors before any synthesis time was spent.
2. **C-synthesis** targeted the `xck26-sfvc784-2LV-c` part at a 200 MHz (5 ns) clock. The resulting `csynth.rpt` provided per-loop latency in cycles, initiation interval, and utilisation of LUTs, DSPs, FFs and BRAM. Every quantitative figure in Section 5 is taken directly from these reports.
3. **On-board execution** deployed the packaged bitstream to the KV260 through the XRT runtime, timing the kernel against an ARM software implementation of the same network compiled into the same executable.

Baseline and bottleneck identification

The unoptimised network was synthesised first with no performance pragmas. The report showed a total latency of 1,202,163 cycles, dominated by two stages: Conv2 at 793,818 cycles and FC1 at 388,979 cycles, each stalled at an initiation interval of nine. The cause was the multi-cycle latency of floating-point addition, which creates a read-after-write dependency on the accumulator and prevents successive iterations from issuing back-to-back. This result reframed the problem: the convolutions were not the sole bottleneck, and the fully-connected layers could not be ignored.

Applied transformations

Four HLS techniques were applied to the compute-bound layers:

- **Fixed-point conversion.** The datapath was changed from 32-bit `float` to `ap_fixed<20,8>`, with a wider `ap_fixed<40,16>` accumulator to guard against overflow. Single-cycle fixed-point addition removed the accumulator recurrence and allowed $\mathrm{II} = 1$; the narrower operands also reduced multiplier cost, which was a prerequisite for the unrolling that followed.
- **Balanced adder tree.** In each convolution, the N products of a filter window ($N = 25$ for Conv1, $N = 75$ for Conv2) are computed in parallel and summed through a binary reduction tree of $\lceil \log_2 N \rceil$ stages, replacing a serial accumulation and sustaining $\mathrm{II} = 1$ on the output-pixel loop.
- **Array partitioning.** Weight and input arrays were split into independent memory banks with `ARRAY_PARTITION` so that the multiple values consumed each cycle could be read simultaneously, rather than serialising on a single BRAM port.
- **Loop unrolling.** The fully-connected MAC loops were unrolled by a tunable factor, replicating multiply-accumulate hardware to process several input elements per cycle. The factor was swept to find the largest value that remained within the device budget.

Only correctness-preserving transformations were used: the fixed-point datapath is active during synthesis, while C-simulation retains floating-point arithmetic, so accuracy could be checked at every step. All weights were compiled into

the kernel as static constants, so no transformation altered the network's numerical behaviour beyond the fixed-point quantisation, whose effect on accuracy was measured directly.

5 Resource and Performance Results

All latency and utilisation figures in this section are C-synthesis estimates for the `xck26` part at 200 MHz, giving a consistent hardware-to-hardware basis for comparison. On-board measurements against the ARM software reference are reported separately at the end.

Optimisation progression

Table 2 traces the design from the sequential baseline to the deployed accelerator. Accelerating only the convolutions (V1) produced a 2.9× speedup, far short of expectation, because FC1 was left untouched and immediately became the dominant stage. Accelerating the fully-connected layers (V2, V3) removed that ceiling and delivered the decisive gains. V3 (FC unroll ×16) is the design deployed on-board and referred to elsewhere in this report as the final design.

	Variant	Latency (μs)	Speedup	LUT	DSP	BRAM	Fits
V0	Baseline (no pragmas)	6010.8	1.0×	21%	10%	47%	✓
V1	Conv accelerated	2056.7	2.9×	36%	17%	38%	✓
V2	Conv + FC (unroll ×8)	83.1	72.3×	58%	19%	52%	✓
V3	Conv + FC (unroll ×16)	69.5	86.5×	77%	22%	58%	✓

Table 2: Optimisation progression (C-synthesis, 200 MHz). Speedup is relative to the V0 baseline. V3 is the design deployed on-board (Section 5.3).

Figure 3 plots the same four latencies on a log scale, making the disproportionate impact of accelerating FC1 (V1 → V2) visible against the comparatively modest gain from accelerating the convolutions alone (V0 → V1).

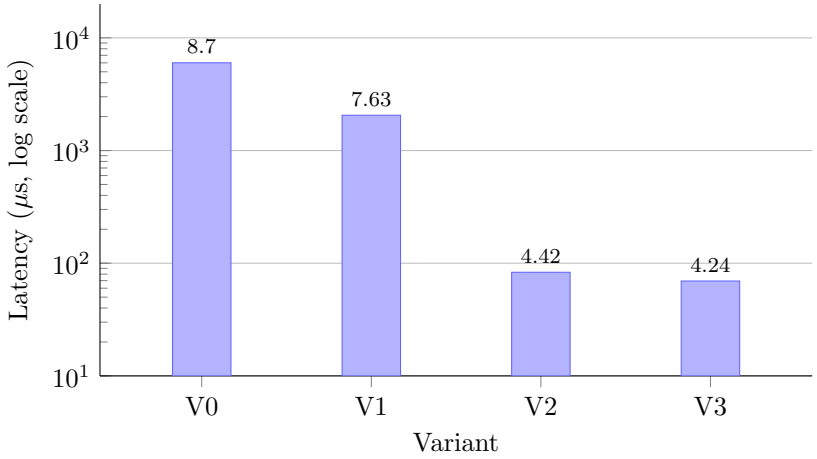

Figure 3: Synthesis-estimated latency across the optimisation progression (log scale). V0→V1 (convolutions only) is a 2.9× reduction; V1→V2 (adding FC acceleration) is a further ∼25× reduction, showing FC1 was the larger of the two bottlenecks.

Fully-connected unroll sweep

Because FC1 was the remaining bottleneck after the convolutions were fixed, its unroll factor was swept to find the best design that still fit the device (Table 3). Factor 32 was faster in isolation but required 114% of available LUTs and therefore could not be placed and routed; factors 20 and 24 also fit within the synthesis LUT budget but were rejected on routability grounds, as discussed in Section 7. Factor 16 was selected as the deployed design for its comfortable routing margin.

A parallel experiment confirmed the same resource ceiling from the other direction: fully parallelising the convolution output channels drove LUT utilisation to 109% while improving latency only marginally (to 70.4 μs), because the convolutions were no longer on the critical path. Both over-budget designs demonstrate that resources spent away from the bottleneck are wasted.

FC unroll factor	Latency (μs)	LUT	DSP	Fits
×8	83.1	58%	19%	✓
×16	69.5	77%	22%	✓
×20	66.5	87%	23%	✓(marginal routing)
×24	65.0	97%	25%	✓(marginal routing)
×32	62.6	114%	27%	×

Table 3: FC unroll factor sweep. Factor 16 was selected for deployment; see Section 7 for the routing-margin rationale behind this choice over the faster but tighter ×20/×24 options.

On-board measurement

The V3 design was deployed to the KV260 and timed against an ARM Cortex-A53 software reference running the identical `cnn()` function, compiled into the same host executable and timed with `chrono::high_resolution_clock`. Both paths process the same 100 MNIST test images end-to-end, from normalised pixel input to argmax output. Table 4 reports the measured per-inference latency and the resulting speedup.

Implementation	Platform	Latency (μs/image)	Speedup
Software baseline (ARM Cortex-A53)	Processing System (PS)	19,392.64	1.0×
V3 FPGA kernel (FC ×16)	Programmable Logic (PL)	168.82	114.87×

Table 4: On-board inference latency averaged over 100 MNIST test images; the 168.82 μs measured figure exceeds the 69.5 μs synthesis estimate due to per-inference XRT and DMA overhead not captured by `csynth.rpt`.

Accuracy

Classification accuracy remained at 100% on the 100-image MNIST test set at every stage of the optimisation, across all design variants and both evaluation datapaths. Table 5 records this per-variant.

Variant	C-simulation	On-board (KV260)
V0 – Baseline (no pragmas)	100%	100%
V1 – Conv accelerated	100%	100%
V2 – Conv + FC (unroll ×8)	100%	100%
V3 – Conv + FC (unroll ×16)	100%	100%

Table 5: Classification accuracy on the 100-image MNIST test set across all design variants. V2 was not deployed on-board; only V3 was taken through to hardware. The `ap_fixed<20,8>` fixed-point format introduced no measurable accuracy degradation relative to the floating-point baseline at any stage.

6 Testing and Verification

Correctness was checked at three levels throughout the project: each individual layer was checked against golden reference vectors, the whole network against labelled classification outcomes, and the hardware was checked against the same software reference it was meant to accelerate. No optimisation was accepted without this verification, so correctness was never lost.

Per-layer unit testing against golden vectors

The network testbench (`cnntest.cpp`) exposes ten test operations selected by a command-line argument: `test_cnn` for the whole network, and one dedicated test per layer, `test_padding_1`, `test_convolution2DRelu_1`, `test_maxPool_1`, `test_padding_2`, `test_convolution2DRelu_2`, `test_maxPool_2`, `test_flattenLayer`, `test_fullyConnected_1`, and `test_fullyConnected_2`. Each calls a single layer function in isolation and compares its output against a static golden-vector array stored per layer in `include/test/oracle/`, covering all 100 images in the bundled MNIST test set.

Each layer test is isolated: a layer's test feeds it the oracle's output for the previous layer, not the output actually produced by the preceding layer in this codebase. For example, `test_fullyConnected_1` calls `fullyConnected_1()` with the oracle's flatten-layer result as input, not this project's own `flattenLayer()` output. This means a bug in one layer can never mask or compound into another layer's test result and so each of the layers is checked independently, so a failure can always be localised to a specific layer.

Two correctness metrics, per component

Two different comparison routines were used across the project, and they enforce different bars for correctness:

- **Per-element absolute-tolerance comparison** (functions `are_similar_3D` / `are_similar_1D`), used for every per-layer oracle test in the full network. Each output element is checked independently against the golden value, and the test fails on the first element that exceeds the tolerance (0.0001 for convolution and fully-connected layers, 0.000001 for max-pooling). Padding, which performs no arithmetic, is checked with the same function called at zero tolerance (`are_equal_3D`), i.e. exact equality. Because this metric fails on the single worst element, it is a strict, non-averaging bar for correctness.
- **RMSE against a double-precision reference**, used for the standalone conv5×5 kernel test that validated the acceleration technique before it was integrated into the full network. A golden convolution is computed in-line using `double` accumulation, and the root-mean-square error across all output elements is thresholded at 10^{-3}. This was a correctness check with deterministically generated test data rather than the MNIST-derived golden vectors, and it achieved an RMSE of approximately 2×10^{-8} in C-simulation.

Whole-network integration testing

`test_cnn` runs the full `cnn()` top-level function on a raw input image, applies argmax to the resulting ten logits, and compares the predicted digit against a golden label. Run across the 100-image test set, this produces the headline accuracy figures reported throughout the project (Section 5): 100% at every stage, in both the floating-point and `ap_fixed<20,8>` datapaths.

Three-stage verification flow

Every transformation was carried through three escalating stages before being accepted, matching the design flow described in Section 4:

1. **C-simulation:** The same C++ sources compiled with `g++ -O2`, with HLS pragmas ignored, run against the oracle vectors and the 100-image test set in seconds. This is the quicker, board-independent check used after every code change.
2. **C-synthesis:** `csynth.rpt` reports cycle latency, per-loop initiation interval, and LUT/DSP/FF/BRAM utilisation for the `xck26` part at 200 MHz, confirming the II, timing closure and resource fit before any hardware build is attempted.
3. **On-board execution:** The bitstream is timed on the KV260 using `chrono::high_resolution_clock`, running a 100 image MNIST accuracy pass and a single image mode, with the ARM software reference compiled into the same binary and timed identically so hardware and software are compared in the same run. For the standalone conv5×5 kernel, 1000 repeated invocations were used to obtain a stable average latency.

Live cross-validation

Beyond the fixed 100-image set, a demonstration application (`draw_demo_app.py`) preprocesses a hand-drawn digit into a 28×28 input and classifies the same input through two separate invocations of `lenet5_host`. `-i` runs the image through the FPGA kernel, while `-s` runs the same image through the ARM software only. Both return the predicted digit, confidence scores, and inference time for both side by side. This shows the design on live input and directly compares hardware and software output.

On-board acceptance criteria

The project's demonstration plan defined explicit pass conditions rather than treating deployment as informally "working": the hardware and software panels of the demo application must agree on every hand-drawn digit; all three deployed builds (baseline, convolution-accelerated, and fully accelerated) must complete the 100-image MNIST batch without error at 100% reported accuracy; and the measured latency progression must fall within the expected range at each stage (approximately 6700, 2300, and 168 μs respectively for V0, V1, and V3). Every criterion was met during the demonstration.

Summary

Across every level, per-layer, whole-network, C-simulation, synthesis-estimate, and on-board, classification accuracy remained at 100% for every optimisation stage, in both the floating-point and fixed-point datapaths, and the standalone convolution kernel's RMSE against a double-precision reference remained at the level of floating-point rounding error. No transformation applied in this project traded away correctness for speed.

7 Key Decisions and Trade-offs

Profile first before intuition

The strongest temptation was to first accelerate the convolutional layers first as we described in our initial plan since, they perform the most arithmetic and are the conventional focus of CNN acceleration. However, from our results we saw that it capped the achievable speedup at 2.9×, because the FC1 accounted for a comparable share of the baseline latency and became the binding constraint the moment the convolutions were sped up. Due to this, we realised profiling before acting was the single decision that most affected the outcome and it argues that measurement should precede optimisation in most cases to see if our intuition also follows.

Fixed-point over floating-point

Adopting `ap_fixed<20,8>` was the enabling decision for the entire project. Its cost is a bounded loss of numerical range and precision relative to 32-bit float; its benefits are single-cycle addition (which removed the II = 9 stall), smaller multipliers (which made high unroll factors affordable), and reduced BRAM pressure. The trade-off was only acceptable because its effect on accuracy could be measured, and it proved to be zero on this task. The wider `ap_fixed<40,16>` accumulator was a deliberate concession of a small amount of fabric to guarantee that summation could not overflow.

Unroll factor and the resource ceiling

Selecting the FC1 unroll factor was an explicit latency-versus-area decision bounded by two distinct limits. Increasing the factor reduces FC1 latency by running more multiply-accumulate operations in parallel, but raises LUT consumption through both the additional MAC units and the wider array partitioning needed to feed them. Sweeping the factor gave the results in Table 6 (repeated from Table 3 for reference).

FC unroll factor	Latency (μs)	LUT	Implementable
×8	83.1	58%	✓
×16	69.5	77%	✓
×20	66.5	87%	marginal
×24	65.0	97%	marginal
×32	62.6	114%	×

Table 6: FC unroll factor sweep (C-synthesis, 200 MHz).

Figure 4 plots latency against LUT utilisation across the sweep, with the two ceilings from Section 7 marked: the ∼85–90% practical routing limit and the 100% absolute synthesis limit. Factor 16 sits comfortably below both; factors 20 and 24 close to diminishing latency returns while approaching the routing ceiling; factor 32 exceeds the device outright.

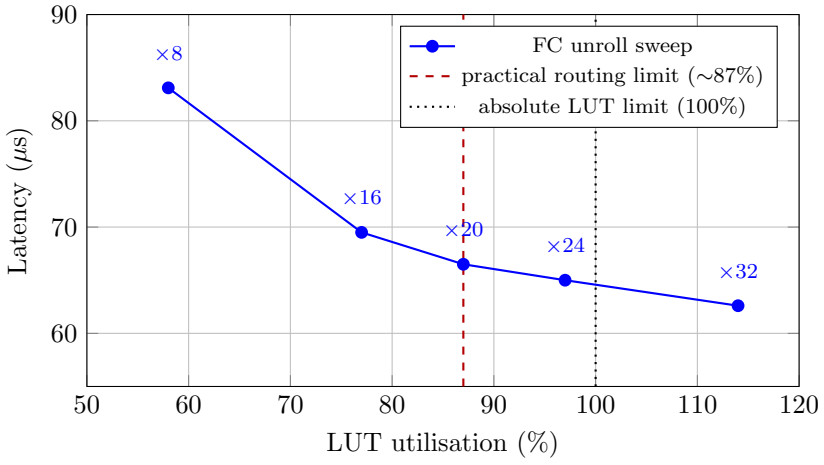

Figure 4: Latency versus LUT utilisation across the FC1 unroll sweep. Factor 16 (deployed) trades a small latency margin for a comfortable buffer below both the practical routing ceiling and the absolute LUT budget.

Two ceilings govern the choice. The first is the absolute synthesis limit: factor 32 requires 114% of available LUTs and cannot be synthesised for the device, the same limit that invalidated the fully channel-parallel convolution at 109%. The second, and more subtle, is routability: a design that fits in synthesis still needs unused fabric for the

router to connect it, and congestion typically prevents place-and-route above roughly 85–90% LUT. Factor 24, at 97%, therefore fits on paper but is unlikely to implement in practice, while factor 20, at 87%, sits at the edge of what is safely routable.

A further consideration is that non-power-of-two factors are less efficient per lane. Cyclic partitioning by a power of two banks the array using free bit-slice addressing, whereas factor 20 requires explicit modulo-and-divide logic to select the bank, so each additional lane above 16 costs disproportionately more LUTs. This, combined with a diminishing latency return, each step buys only a few percent as the fixed convolution, pooling and buffer-load overhead comes to dominate, led us to select factor 16 as the deployed design for its comfortable routing margin, with factor 20 retained as a faster alternative where the additional place-and-route effort is justified.

The governing principle is that an FPGA has both an absolute resource limit and a lower practical routing limit; a design that exceeds either has no deployable performance at all, regardless of its synthesis-estimated latency, and that's why we chose 16 for our unroll factor instead.

Latency versus throughput

Our design was optimised for latency instead of throughput, which suited our interactive use case within the demonstration, where a user submits one digit and waits for the result. The two are independent: throughput can be raised without improving single-image latency by overlapping successive images, so that one image begins in the early layers while the previous image is still completing the later ones. We applied `#pragma HLS DATAFLOW` to do exactly this, and in synthesis it allowed a new image to start every 23.5 μs rather than every 59.4 μs, roughly 2.5× the throughput, while single-image latency was unchanged, at the cost of additional BRAM for the buffers inserted between stages. Choosing between the two is a genuine architectural decision rather than a tuning parameter, and depends on whether the target workload is dominated by single-image responsiveness or by bulk processing.

Weights on-chip over streamed

Because the network is small (approximately 45,000 parameters), all weights were compiled into on-chip BRAM rather than streamed from DDR. This eliminated weight-transfer latency entirely and left the AXI bus carrying only the input image and output logits, at the cost of committing BRAM that a larger model could not have afforded. The decision was appropriate for this network but does not generalise to models whose parameters exceed on-chip capacity.

8 Future Optimisation Opportunities

Three directions stood out as the next steps, each targeting a part of the design that was deliberately left unoptimised or untested within the project's timeline.

Accelerating FC2

FC2 (the 147×10 output layer) was left without the `ARRAY_PARTITION/UNROLL` treatment applied to FC1 in Section 4, and currently runs as plain sequential C++. This is a smaller target than FC1 by an order of magnitude (1,470 multiply-accumulates versus FC1's $294 \times 147 \approx 43{,}000$), which makes it a strong candidate for full unrolling rather than a partial factor sweep. All ten outputs could possibly be computed in parallel without approaching the LUT ceiling that bounded FC1's unroll factor in Section 7. Because FC2's contribution to total latency is small in absolute terms, the main benefit would not be raw speed but removing the last unaccelerated arithmetic stage from the pipeline, which also simplifies the latency model of the design as a whole.

Binary input thresholding

The input image is currently carried as an 8-bit greyscale value (0 to 255, converted to `ap_fixed<20,8>`) all the way into Conv1. For the hand-drawn digit input, thresholding each pixel to a strict binary value (0 or 1) before it reaches the kernel would change every multiplication in Conv1's 25-tap filter from a full fixed-point multiply to a conditional select between a weight value and zero, which is significantly cheaper in LUT fabric than a general multiplier. Since Conv1 is one of only two convolution layers and the only one touching raw pixel data, this could free resource budget for the other optimisations in this section without touching Conv2 or the fully-connected layers at all. The trade-off is that MNIST's training data is anti-aliased greyscale, not binary, so this would need validation against the existing 100-image accuracy benchmark (Section 6) to confirm classification accuracy holds under thresholded input.

Deploying dataflow to the board

Applying `#pragma HLS DATAFLOW` across the nine pipeline stages was shown in synthesis (Section 7) to raise throughput by roughly 2.5×, letting a new image begin every 23.5 μs rather than every 59.4 μs, at unchanged single-image latency. This was never taken past C-synthesis to an on-board build. Deploying it would confirm the estimate holds under

real XRT/DMA overhead and would quantify the BRAM cost of the inter-stage buffers it introduces, which synthesis reports but which was never weighed against the 58% BRAM utilisation of the deployed design.

9 Team Delegation and Framework

Table 7 summarises each team member's contributions. Work was structured as a milestone sequence rather than fully parallel tracks, since the pipeline had a hard dependency: HLS implementation could not be validated without golden reference vectors, so Kaira's test vectors and weight extraction were produced first, followed by Shuruthy's HLS layer development against those vectors, and finally Gilbert's integration, FC optimisation, and on-board deployment.

Shuruthy Dhushiyandan	**Kaira Dumasia**	**Gilbert Tong**
• Built the conv5×5 HLS kernel from scratch • Switched from float to `ap_fixed<20,8>` for II=1 pipelining • Implemented parallel 25-multiplier structure with balanced adder tree • Ran C-simulation at each step to validate correctness	• Simulated bit widths to preserve accuracy through fixed-point conversion • Produced golden test vectors for C-sim validation • Extracted and formatted trained weights for HLS use • Curated and prepared the final demo	• Integrated optimised conv5×5 into the full LeNet architecture • Optimised FC layers with `UNROLL` directive, sweeping ×8, ×16, ×32 • Built the Vivado block design and hardware-software integration • Deployed and tested on the Zynq board end to end

Table 7: Team progress and individual contributions.

The tasks differs from the initial project plan, in which fully connected optimisation was not explicitly assigned and was marked as "extend to remaining layers if time allows". instead Gilbert did it alongside integration, and it turned out to be a large contributor to the final speedup (Section 5).

The same profile-transform-verify loop used for the hardware design (Section 4) was also used with team members: each stage's output (vectors, then HLS layers, then the integrated design) had to pass its own verification gate before the next stage began, so correctness was never assumed to carry over from one person's work to the next. However, looking back now, given that the per-layer tests treat each layer independently (Section 6), HLS optimisation and integration could possibly have proceeded in parallel on separate layers once the single conv proof of concept was validated, rather than sequentially.

Generative AI (Claude) was used during the project to help interpret Vitis HLS pragma semantics, debug initiation-interval violations, and resolve XRT deployment issues.

10 Conclusion

This project demonstrated that a systematic, profiling-driven approach to CNN acceleration on FPGA can deliver substantial performance gains without sacrificing accuracy or exceeding resource constraints.

The central finding was that parallelism must be directed at the true bottleneck. Accelerating the convolutional layers alone produced only a 2.9× speedup, as FC1 immediately became the dominant constraint on runtime. Once FC1 was also accelerated using fixed-point arithmetic and 16-wide parallel MAC units, the overall measured speedup against the ARM software baseline reached 114.87× on the board. Attempting to further parallelise the convolutional layers pushed LUT utilisation to 109%, preventing implementation and yielding a slower result. This confirms that committing resources to a layer that is already fast provides minimal return.

Fixed-point conversion was fundamental to achieving these gains. Switching to `ap_fixed<20,8>` eliminated the accumulator dependency that caused II = 9 at baseline, enabling II = 1 pipelining and making high unroll factors viable within the device resource budget. Classification accuracy was maintained at 100% across every stage of optimisation, confirming that the chosen fixed-point format introduced no numerical degradation.

Overall, the project established that over 100× speedup against the ARM software baseline is achievable on embedded FPGA hardware through careful profiling and targeted application of HLS pragmas, without compromising correctness or exceeding hardware resource limits.

References

[1] Advanced Micro Devices (AMD). *Kria KV260 Vision AI Starter Kit — Data Sheet and User Guide*, 2024. https://www.amd.com/en/products/system-on-modules/kria/k26/kv260-vision-starter-kit.html.

[2] Advanced Micro Devices (AMD). *Vitis High-Level Synthesis User Guide (UG1399)*, 2025. https://docs.amd.com/r/en-US/ug1399-vitis-hls.

[3] Advanced Micro Devices (AMD). *Xilinx Runtime (XRT) Documentation*, 2025. https://xilinx.github.io/XRT/.

[4] David de Andrés and Juan Carlos Ruiz. Lenet5FloatHLS: A LeNet-5 CNN in C++ for High-Level Synthesis. Fault-Tolerant Systems, Instituto ITACA, Universitat Politècnica de València, 2023. MIT License. Available: https://git.upv.es/defadas/Lenet5FloatHLS.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[6] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. The MNIST database of handwritten digits. http://yann.lecun.com/exdb/mnist/, 1998.